Canvas Helper AI Assistant
Requirements Traceability Matrix

Team: Team 16 · Course: COSC 499 · Date: August 2026
Final Delivery — Requirements Implementation & Validation Traceability

1. Purpose

This matrix demonstrates that every documented User Requirement (UR) and Functional Requirement (FR) for the Canvas Helper AI Assistant has been implemented and validated. Each row links a requirement to concrete implementation surfaces and verification evidence from automated tests, end-to-end checks, and manual workflow validation.

2. Summary

Requirement set	Count	Result
Administrator user requirements (UR1.1–UR1.7)	7	**7 Satisfied / 0 Unmet**
Instructor user requirements (UR2.1–UR2.29)	29	**29 Satisfied / 0 Unmet**
Functional requirements (FR1–FR13)	All listed items	**All Satisfied**
Overall user requirements	36	**36 Satisfied / 0 Unmet**

3. User Requirements Traceability

UR1 covers administrator capabilities; UR2 covers instructor capabilities. Validation evidence cites representative automated tests and workflow checks in the repository.

3.1 Administrator requirements (UR1.1–UR1.7)

ID	Requirement	Implementation evidence	Validation evidence	Status
UR1.1	Secure administrator login and access to administrative functions	Admin auth UI/API; JWT sessions; RBAC	test_admin_auth.py; e2e admin.spec; manual admin login	**Satisfied**
UR1.2	Create, update, enable, disable, and manage instructor accounts	Admin instructor management pages/services	test_admin_management.py; InstructorManagementPage tests	**Satisfied**
UR1.3	Revoke instructor access when required	Revoke Canvas access / disable account flows	test_admin_service_unit.py revoke tests; admin UI revoke flow	**Satisfied**
UR1.4	Monitor system health and operational status	System Health dashboard + health services	test_health.py; SystemHealthPage tests; /health checks	**Satisfied**
UR1.5	Review audit logs and security-related events	Audit log viewer + audit logging	test_audit_service_unit.py; AuditLogsPage tests	**Satisfied**
UR1.6	Review failed Canvas and AI operations	Failed Operations page + failed_operations	test_failed_operations.py; FailedOperationsPage tests	**Satisfied**
UR1.7	Troubleshoot without accessing sensitive credentials	Troubleshooting diagnostics with credential masking	test_troubleshoot_*.py; TroubleshootingPage tests	**Satisfied**

3.2 Instructor requirements (UR2.1–UR2.29)

ID	Requirement	Implementation evidence	Validation evidence	Status
UR2.1	Secure account creation, authentication, and application access	Register/login/JWT/protected routes	test_auth.py; test_auth_service_unit.py; e2e instructor auth	**Satisfied**
UR2.2	Securely connect a Canvas API token	Connect Canvas UI + encrypted token storage	test_canvas*.py token flows; ConnectCanvasPage tests	**Satisfied**
UR2.3	Replace or remove a stored Canvas API token	Settings/connect replace & remove flows	auth canvas-token tests; CanvasConnectionTab tests	**Satisfied**
UR2.4	View Canvas courses for the connected token	Courses list + Canvas sync	test_canvas.py list/sync; InstructorCoursesPage tests	**Satisfied**
UR2.5	Select an active course	Active course preference + sidebar selector	active-course API tests; ActiveCourse* component tests	**Satisfied**

ID	Requirement	Implementation evidence	Validation evidence	Status
UR2.6	View course info, students, assignments, and submissions	Course tabs + Canvas retrieval services	canvas/course page tests; submissions/assignments tests	**Satisfied**
UR2.7	Analyze missing submissions	Missing submissions analysis UI/service	test_missing_submissions.py; analyse-missing chat tests	**Satisfied**
UR2.8	Analyze late submissions	Late submissions analysis UI/service	test_late_submissions.py; submissions analysis UI tests	**Satisfied**
UR2.9	Search and filter course information	Tables search/filter/pagination	Frontend table/filter unit tests; course list UI tests	**Satisfied**
UR2.10	Access a dashboard summarizing course activity	Instructor dashboard overview	test_instructor_dashboard.py; InstructorDashboardPage tests	**Satisfied**
UR2.11	Interact using natural-language requests	AI Assistant chat + LLM services	test_chat_endpoint.py; InstructorAssistantPage tests	**Satisfied**
UR2.12	Ask questions about course data	Read-only assistant/Canvas context tools	test_read_actions.py; chat read-path tests	**Satisfied**
UR2.13	Receive explanations of proposed actions before execution	Preview/explanation cards before confirm	preview/PendingActionControls tests; chat confirm tests	**Satisfied**
UR2.14	Create Canvas assignments	Assignment create form + assistant create_assignment	test_assignment*.py; AssignmentCreatePage tests	**Satisfied**
UR2.15	Update assignment details and dates	Assignment update + due-date normalization	test_due_date.py; assignment update tests	**Satisfied**
UR2.16	Create Canvas quizzes	Quiz create workflow + validation	test_quiz_*.py; InstructorQuizCreatePage tests	**Satisfied**
UR2.17	List submissions for assignments	Submissions pages + Canvas submissions API	test_canvas.py submissions; CourseSubmissionsPage tests	**Satisfied**
UR2.18	Draft Canvas messages to selected students	Messages compose/draft + templates	test_messages*.py; InstructorMessagesPage tests	**Satisfied**
UR2.19	Review and edit messages before sending	Message editor/preview before confirm	message draft/preview component tests	**Satisfied**
UR2.20	Draft feedback comments for selected submissions	Feedback create/draft workflow	test_feedback_*.py; FeedbackCreatePage tests	**Satisfied**
UR2.21	Review and approve feedback before posting	Feedback review + confirm gate	feedback validation/integration tests	**Satisfied**
UR2.22	Modify grades through approved workflows	Grade preview/submit + strong CONFIRM	test_grade_management.py; grades UI tests	**Satisfied**
UR2.23	Preview Canvas-modifying actions before execution	Dry-run / write preview services	test_write_preview.py; DryRunPreviewPage tests	**Satisfied**
UR2.24	Explicitly approve Canvas-modifying actions before execution	Confirm controls; no auto-write on chat turn	test_chat_confirm.py; action classification tests	**Satisfied**
UR2.25	Cancel a proposed action before execution	Cancel path leaves Canvas unchanged	chat cancel tests; e2e preview cancel path	**Satisfied**
UR2.26	View a history of previous actions	Workflow/action history UI	test_workflow_history.py; WorkflowHistoryPage tests	**Satisfied**
UR2.27	View previous assistant requests and outcomes	Conversation/history + workflow status	history/chat status tests; assistant history UI	**Satisfied**
UR2.28	Determine whether a previous action can be reversed	Reversal metadata + undo for supported creates	test_workflow_undo.py; reversal metadata tests	**Satisfied**
UR2.29	Clear feedback when requests cannot be completed safely	Clarifications, errors, out-of-scope refusals	test_error_handling.py; assistant safety tests	**Satisfied**

4. Functional Requirements Traceability

Functional requirements are grouped by system area. Status is Satisfied when both an implementation path and verification evidence exist.

FR1 — Authentication, Accounts & Canvas Credentials

ID	Requirement	Implementation evidence	Validation evidence	Status
FR1.1	Instructor login required before Canvas functionality	Auth + route guards	auth/RBAC tests; ProtectedRoute tests	**Satisfied**

ID	Requirement	Implementation evidence	Validation evidence	Status
FR1.2	Administrators manage instructor accounts	Admin users service/UI	admin management tests	**Satisfied**
FR1.3	Administrators can revoke instructor access	Revoke/disable flows	revoke access unit/API tests	**Satisfied**
FR1.4	Instructors enter/replace/remove Canvas tokens	Connect Canvas / settings	canvas token API + UI tests	**Satisfied**
FR1.5	Canvas API tokens encrypted before storage	Fernet encryption	security/canvas connection tests	**Satisfied**
FR1.6	Canvas API tokens validated before use	Token validation service	canvas validation tests	**Satisfied**
FR1.7	Stored tokens never exposed after submission	Masked UI; no token echo	UI/security tests; redaction checks	**Satisfied**
FR1.8	Canvas ops use authenticated instructor token	Per-instructor connections	canvas ownership tests	**Satisfied**
FR1.9	No cross-instructor token access	Ownership checks	cross-instructor rejection tests	**Satisfied**
FR1.10	Logout and session expiration supported	Sessions + revoke on logout/password change	session/logout tests	**Satisfied**

FR2 — Course Dashboard & Canvas Data Access

ID	Requirement	Implementation evidence	Validation evidence	Status
FR2.1	Retrieve instructor courses	Canvas course sync/list	test_canvas.py course list/sync	**Satisfied**
FR2.2	Select active course	Active-course preference API/UI	active-course API + UI tests	**Satisfied**
FR2.3	Retrieve students for selected course	Students service/pages	student retrieval tests	**Satisfied**
FR2.4	Retrieve assignments for selected course	Assignments service/pages	assignment list tests	**Satisfied**
FR2.5	Retrieve submissions for assignments	Submissions service/pages	submission list tests	**Satisfied**
FR2.6	Dashboard course-level summaries	Instructor/course overview	dashboard/overview tests	**Satisfied**
FR2.7	Assignment-level info and submission status	Course assignments/overview	course overview/submission tests	**Satisfied**
FR2.8	Identify submitted/missing/late/graded where available	Submission analysis	missing/late analysis tests	**Satisfied**
FR2.9	Suggested actions / workflow shortcuts	Dashboard quick actions	dashboard UI tests	**Satisfied**
FR2.10	Canvas is source of truth for course/assignment data	Sync model; local cache only	canvas sync/source-of-truth tests	**Satisfied**

FR3 — Natural-Language Assistant

ID	Requirement	Implementation evidence	Validation evidence	Status
FR3.1	Chat-style NL interface	Assistant page + ChatWindow	ChatWindow/assistant page tests	**Satisfied**
FR3.2	Support approved Canvas course-management workflows	Allowlisted actions	action schema/classification tests	**Satisfied**
FR3.3	Interpret requests into structured actions	Chat/LLM structured output	chat endpoint/JSON extraction tests	**Satisfied**
FR3.4	Request clarification when incomplete/ambiguous	Clarification responses	chat clarification tests	**Satisfied**
FR3.5	Display interpreted intent before execution	Proposal/preview cards	ActionCard/preview tests	**Satisfied**
FR3.6	Support listed read/write workflows	Tool schemas + UI handoffs	chat + workflow integration tests	**Satisfied**
FR3.7	Explain unsupported requests and prevent execution	out_of_scope / refusal paths	assistant safety/out-of-scope tests	**Satisfied**
FR3.8	Must not invent Canvas data not retrieved from Canvas	Grounding / ID scrub	Feature 13 grounding tests	**Satisfied**
FR3.9	Scope to active/explicit course context	Course resolution priority	course resolution/chat binding tests	**Satisfied**

FR4 — Assignment & Quiz Management

ID	Requirement	Implementation evidence	Validation evidence	Status
FR4.1	Create Canvas assignments	Assignment create services/UI	assignment create/integration tests	**Satisfied**
FR4.2	Update assignment details including dates	Assignment update + due_date	assignment update + due_date tests	**Satisfied**
FR4.3	Create Canvas quizzes	Quiz create services/UI	quiz draft/integration tests	**Satisfied**
FR4.4	Validate assignment/quiz data before Canvas submit	Validation services	assignment/quiz validation tests	**Satisfied**

ID	Requirement	Implementation evidence	Validation evidence	Status
FR4.5	Require preview and confirmation for create/modify	Write preview + confirm	preview/confirm integration tests	**Satisfied**

FR5 — Submission Analysis & Grading

ID	Requirement	Implementation evidence	Validation evidence	Status
FR5.1	Identify missing submissions	Missing analysis service/UI	test_missing_submissions.py	**Satisfied**
FR5.2	Identify late submissions	Late analysis service/UI	test_late_submissions.py	**Satisfied**
FR5.3	Display submission summaries and affected students	Analysis tables/summaries	submissions analysis UI/API tests	**Satisfied**
FR5.4	Support grade modification via approved operations	Grade management services	test_grade_management.py	**Satisfied**
FR5.5	Grade modifications require preview and confirmation	Grade preview + CONFIRM	grade preview/submit tests	**Satisfied**

FR6 — Messaging & Feedback

ID	Requirement	Implementation evidence	Validation evidence	Status
FR6.1	Generate draft Canvas messages	Message draft services/UI	message draft/service tests	**Satisfied**
FR6.2	Review/edit messages before sending	Message editor	message editor/component tests	**Satisfied**
FR6.3	Message delivery requires preview and confirmation	Send confirm gate	message send validation tests	**Satisfied**
FR6.4	Generate feedback comments for submissions	Feedback draft workflow	feedback draft tests	**Satisfied**
FR6.5	Feedback posting requires preview and confirmation	Feedback confirm gate	feedback validation/integration tests	**Satisfied**
FR6.6	Messaging and feedback recorded in action history	Workflow/history logging	history + feedback audit tests	**Satisfied**

FR7 — Dry-Run Preview & Confirmation

ID	Requirement	Implementation evidence	Validation evidence	Status
FR7.1	Every Canvas-modifying action generates dry-run preview	write_preview / confirm flow	test_write_preview.py; preview e2e	**Satisfied**
FR7.2	Preview summarizes action and affected resources	Preview panel cards	PreviewPanel component tests	**Satisfied**
FR7.3	Preview identifies reversibility	Reversibility status	reversal metadata tests	**Satisfied**
FR7.4	Explicit confirm required before Canvas write	Confirm controls	test_chat_confirm.py	**Satisfied**
FR7.5	Users can cancel proposed actions	Cancel path	cancel/chat cancel tests	**Satisfied**
FR7.6	Classify actions as read-only, modifying, or irreversible	Action classification	test_action_classification.py	**Satisfied**

FR8 — Action History & Limited Reversal

ID	Requirement	Implementation evidence	Validation evidence	Status
FR8.1	Maintain user-visible action history	History pages/API	test_workflow_history.py; history UI	**Satisfied**
FR8.2	History includes request, action, status, timestamp, affected resources	Workflow history model	history API/schema tests	**Satisfied**
FR8.3	Maintain history of assistant requests and associated actions	Conversation/workflow linkage	chat/history integration tests	**Satisfied**
FR8.4	Support limited reversal where safe/practical	Undo for supported creates	test_workflow_undo.py	**Satisfied**
FR8.5	Explain when reversal is unavailable	Reversal metadata messaging	test_action_reversal_metadata.py	**Satisfied**

FR9 — AI Validation & MCP Integration

ID	Requirement	Implementation evidence	Validation evidence	Status
FR9.1	AI model must not directly access Canvas APIs	LLM → backend tools only	assistant safety / dispatcher tests	**Satisfied**

ID	Requirement	Implementation evidence	Validation evidence	Status
FR9.2	Canvas operations routed through backend and MCP/tool layer	Dispatcher + optional MCP	test_mcp_*.py; dispatcher tests	**Satisfied**
FR9.3	Backend validates AI-generated actions before execution	Validation before confirm/execute	validation + chat confirm tests	**Satisfied**
FR9.4	Reject malformed/unsupported/unauthorized/unsafe actions	Allowlists + validators	error-handling / safety tests	**Satisfied**
FR9.5	MCP/tool layer exposes only approved Canvas operations	MCP allowlist/host controls	MCP server/tool allowlist tests	**Satisfied**
FR9.6	Dockerized MCP server or equivalent tool-execution layer	app/mcp + compose	compose config; MCP deployment docs/tests	**Satisfied**

FR10 — Security & Privacy

ID	Requirement	Implementation evidence	Validation evidence	Status
FR10.1	Encrypt Canvas tokens and AI API keys at rest	Fernet / encrypted columns	security/encryption tests	**Satisfied**
FR10.2	Credentials never in logs, errors, responses, repos, or UI	Redaction + masking	secret_redaction / security tests	**Satisfied**
FR10.3	Student information only for authorized users	AuthZ / course ownership	RBAC + ownership rejection tests	**Satisfied**
FR10.4	Minimize student-identifiable info sent to AI	Student context minimization	student_context / privacy tests	**Satisfied**
FR10.5	Validate all Canvas-modifying actions before forwarding	Backend validation gate	write validation/confirm tests	**Satisfied**

FR11 — Reliability & Error Handling

ID	Requirement	Implementation evidence	Validation evidence	Status
FR11.1	Meaningful user-facing error messages	Error classification + UI alerts	test_error_handling.py; ErrorAlert tests	**Satisfied**
FR11.2	Safely handle Canvas API failures	Failure recording; no fake success	canvas write failure tests	**Satisfied**
FR11.3	Safely handle incomplete/unavailable Canvas data	Graceful empty/partial states	canvas incomplete-data tests	**Satisfied**
FR11.4	Prevent duplicate Canvas-modifying execution	Idempotency keys / write guards	test_canvas_idempotency_*.py	**Satisfied**

FR12 — Administration & Deployment

ID	Requirement	Implementation evidence	Validation evidence	Status
FR12.1	Administrator operational health information	System health page	health service/UI tests	**Satisfied**
FR12.2	Monitor failed Canvas and AI operations	Failed operations page	failed_operations tests	**Satisfied**
FR12.3	Web-based application	React + FastAPI + nginx compose	compose smoke; e2e web flows	**Satisfied**
FR12.4	Docker Compose for local deployment	docker-compose.yml / Makefile	make up / deployment guide verification	**Satisfied**
FR12.5	Developer setup and deployment documentation	docs/final deployment + developer guides	docs package review	**Satisfied**
FR12.6	Separated frontend, backend, DB, Canvas, MCP components	Repository architecture	architecture docs + compose layout	**Satisfied**

FR13 — Scope Constraints

ID	Requirement	Implementation evidence	Validation evidence	Status
FR13.1	No autonomous Canvas modifications without approval	Confirm-before-write; no write on normal chat turn	NORMAL_TURN_ALLOWED_ACTIONS tests; confirm tests	**Satisfied**
FR13.2	No Canvas API execution bypassing backend validation	Allowlisted tools only	dispatcher/allowlist tests	**Satisfied**
FR13.3	No institution-wide Canvas administration	Instructor-scoped token/course	scope/RBAC tests	**Satisfied**
FR13.4	No unrestricted deletion of Canvas resources	Deletes limited to undo paths	undo/path allowlist review tests	**Satisfied**
FR13.5	No arbitrary user-defined Canvas API calls	No free-form Canvas proxy	MCP host allowlist + tool schema tests	**Satisfied**

5. Milestone Coverage (Implementation Path)

Requirements were delivered incrementally across project milestones:

Milestone	Focus	Requirements advanced
M1 — Proof of Life	FE/BE foundation, auth, Canvas, local AI, Docker	Foundation for UR2.1–2.3, 2.11–2.12; FR1–FR3, FR12–FR13
M2 — MVP Demo	Canvas data, assistant, assignments, messaging, preview, history	UR1.1–1.3; UR2.1–2.15, 2.18–2.19, 2.23–2.29; FR1–FR5, FR7–FR8, FR11
M3 — Testing Event 1	MVP validation, coverage growth, usability fixes	Validated MVP UR/FR workflows
M4 — Testing Event 2	Quizzes, feedback, grades, reversal + regression	UR2.16, 2.20–2.22, 2.28; FR4, FR6, FR8
M5 — Final Demo	Admin monitoring, LLM providers, security, reliability, MCP	UR1.4–1.7; FR9–FR12
M6 — Final Delivery	Documentation, deployment package, final verification	FR12.5–12.6 documentation and packaging

6. Evidence Sources

- Automated backend tests under `test/backend/` (`make test`, optional `make test-fuzz`).
- Automated frontend tests under `test/frontend/` (Vitest coverage via `make test`).
- Browser end-to-end checks under `test/e2e/` (`make test-e2e`).
- Manual workflow validation during Testing Events 1 and 2 and final demo preparation.
- Final documentation package under `docs/final/` for setup, architecture, and operator guidance.

7. Conclusion

All administrator requirements (UR1.1–UR1.7), instructor requirements (UR2.1–UR2.29), functional requirement groups (FR1–FR13), and scope constraints are implemented in the delivered system and verified through the evidence cited above. No unmet requirement-level gaps remain at final submission. Residual product limitations are intentional scope constraints (FR13), not unmet requirements.